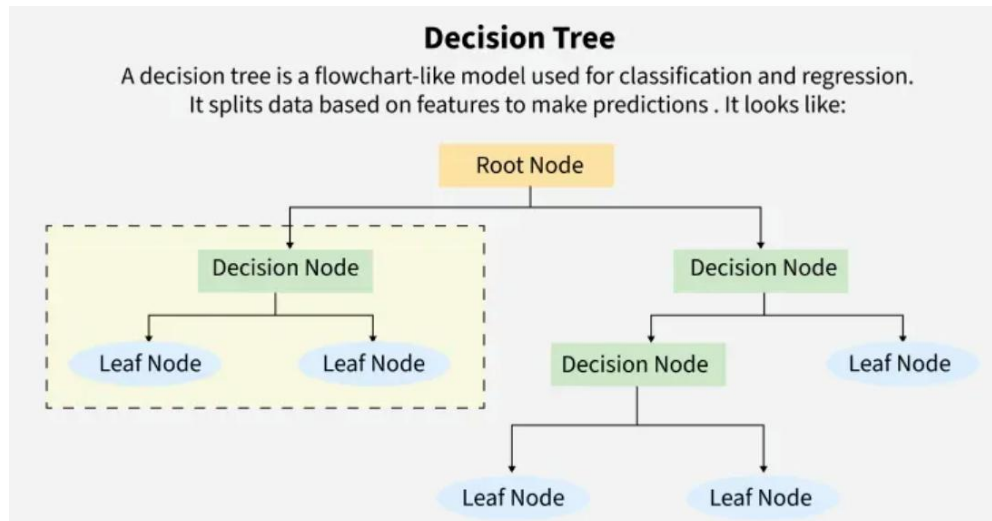


Decision Tree Algorithm: A decision tree is a supervised learning algorithm used for both classification and regression tasks. It has a hierarchical tree structure which consists of a root node, branches, internal nodes and leaf nodes. It works like a flowchart help to make decisions step by step where:

- **Internal nodes:** Represents the entire dataset, split by the best attribute.
- **Branches:** Show the outcome of a test and connect nodes.
- **Leaf nodes:** Represents the final class/decision.
- **Decision Node:** Represents a test on an attribute, with multiple branches.

Decision trees are widely used due to their interpretability, flexibility and low preprocessing needs.



Working of a Decision Tree:

- 1.Start with the entire dataset at the root node.
- 2.Select the best attribute for splitting using a splitting criterion.
- 3.Divide the dataset into subsets based on the chosen attribute.
- 4.Create decision nodes for subsets that are not pure and leaf nodes for subsets that are pure.
- 5.Repeat the process recursively for each decision node.
- 6.Stop when all records in a node belong to the same class, when no attributes remain, or when a stopping condition is met.
- 7.The final decision tree represents a hierarchy of decisions where each path from root to leaf corresponds to a classification or prediction rule.

Splitting Criteria:

1.Information Gain (ID3 Algorithm):

Based on **Entropy**, which measures impurity or randomness.

Information Gain calculates the reduction in entropy after splitting on an attribute.

The attribute with the **highest Information Gain** is chosen.

2.Gini Index (CART Algorithm):

Measures the probability of incorrectly classifying a randomly chosen record.

A node is pure if all records belong to a single class (Gini = 0).

The attribute that produces the **lowest Gini Index after splitting** is chosen.

3.Variance Reduction (Regression Trees):

Used when the target variable is continuous (numerical).

Measures how much the variance of the target decreases after the split.

The split with the maximum variance reduction is chosen.

Advantages:

- 1.Simple and Easy to Understand.
- 2.Handles Both Types of Data.
- 3.Handles Missing Values.

Disadvantages:

- 1.Overfitting.
- 2.Instability and High Variance.
- 3.Bias towards features with many levels.

Role of Entropy:

- 1.Entropy is a measure of impurity, disorder, or uncertainty in a dataset.
- 2.It is used in Information Gain (ID3 algorithm) to decide the best attribute for splitting.
 - If all samples in a dataset belong to the same class, the entropy is 0 (perfectly pure).
 - If the samples are equally distributed among classes, the entropy is maximum (completely impure).
- 3.**Ex:** In real life, entropy is used in decision trees for problems like **loan approval, medical diagnosis, fraud detection, or customer churn prediction**, where the system measures impurity (uncertainty) in the dataset and splits on attributes to make purer groups, improving decision-making.

Information Gain:

- 1.Information Gain is a splitting criterion used in decision trees.
- 2.It measures how much uncertainty (entropy) in the dataset is reduced after splitting on an attribute.
 - High Information Gain → the attribute provides a better split.
 - Low Information Gain → the attribute is less useful.

CART (Classification and Regression Tree): CART is a decision tree algorithm that is used for both classification and regression tasks. It is a supervised learning algorithm that learns from labelled data to predict unseen data.

Splitting Criteria in CART:

- 1.**Classification Trees:** Uses Gini Index to choose the best attribute.
- 2.**Regression Trees:** Uses Variance Reduction to choose the best split for numerical outcomes.

Working of CART:

- 1.Start with the full dataset at the root node.
- 2.Evaluate all possible splits for all attributes using the appropriate criterion (Gini or Variance Reduction).
- 3.Select the split that produces the **purest child nodes**.
- 4.Repeat the process recursively for each child node.
- 5.Stop when a stopping condition is met (e.g., minimum samples per node, maximum depth).
- 6.The resulting tree is **binary**, with internal nodes representing tests and leaves representing class labels or predicted values.

Key Points:

- Produces **binary trees only**.
- Can handle **both classification and regression** problems.
- Uses **Gini Index** for classification and **Variance Reduction** for regression.
- Can be **pruned** to avoid overfitting.

Random Forest: Random Forest is a machine learning algorithm that uses many decision trees to make better predictions. Each tree looks at different random parts of the data and their results are combined by voting for classification or averaging for regression which makes it as ensemble learning technique. This helps in improving accuracy and reducing errors.

Working of Random Forest:

1. From the training dataset, generate **multiple bootstrap samples** (random sampling with replacement).
2. For each sample, **build a decision tree** using a random subset of features at each split (this adds randomness and reduces correlation between trees).
3. Each tree makes a **prediction** independently:
 - **Classification:** Each tree votes for a class, and the majority vote is chosen.
 - **Regression:** The average of all tree predictions is taken.
4. Combine all tree predictions to get the **final result**.

Assumptions of Random Forest:

1. **Each tree makes its own decisions:** Every tree in the forest makes its own predictions without relying on others.
2. **Random parts of the data are used:** Each tree is built using random samples and features to reduce mistakes.
3. **Enough data is needed:** Sufficient data ensures the trees are different and learn unique patterns and variety.
4. **Different predictions improve accuracy:** Combining the predictions from different trees leads to a more accurate final result.

Key Features of Random Forest:

1. **Handles Missing Data:** It can work even if some data is missing so you don't always need to fill in the gaps yourself.
2. **Shows Feature Importance:** It tells you which features (columns) are most useful for making predictions which helps you understand your data better.
3. **Works Well with Big and Complex Data:** It can handle large datasets with many features without slowing down or losing accuracy.
4. **Used for Different Tasks:** You can use it for both **classification** like predicting types or labels and **regression** like predicting numbers or amounts.

Classification Metrics: Classification metrics are ways to measure how good a model is at predicting classes correctly. They show how many predictions are right or wrong.

Predicted \ Actual	Positive (P)	Negative (N)
Positive (Predicted Yes)	True Positive (TP)	False Positive (FP)
Negative (Predicted No)	False Negative (FN)	True Negative (TN)

1. **True Positive (TP):** Model predicts positive, and it is actually positive.
2. **False Positive (FP):** Model predicts positive, but it is actually negative. (Type I error).
3. **True Negative (TN):** Model predicts negative, and it is actually negative.
4. **False Negative (FN):** Model predicts negative, but it is actually positive. (Type II error).

Example of E-Mail:

1.True Positive (TP): Email is actually Spam, and the model predicts Spam.

- **Example:** Email “Win \$1000 now!” → correctly classified as Spam.

2.False Positive (FP): Email is actually Not Spam, but the model predicts Spam.

- **Example:** Email “Meeting agenda attached” → incorrectly marked as Spam (Type I error).

3.True Negative (TN): Email is actually Not Spam, and the model predicts Not Spam.

- **Example:** Email “Project update” → correctly classified as Not Spam.

4.False Negative (FN): Email is actually Spam, but the model predicts Not Spam.

- **Example:** Email “Claim your prize” → missed by the filter, classified as Not Spam (Type II error).

Overfitting: Overfitting happens when a model learns too much from the training data, including details that don’t matter (like noise or outliers).

Reasons for Overfitting:

- 1.High variance and low bias.
- 2.The model is too complex.
- 3.The size of the training data.

Underfitting: Underfitting is the opposite of overfitting. It happens when a model is too simple to capture what’s going on in the data.

Reasons for Underfitting:

- 1.The model is too simple, so it may be not capable to represent the complexities in the data.
- 2.The input features which is used to train the model is not the adequate representations of underlying factors influencing the target variable.
- 3.The size of the training dataset used is not enough.
- 4.Excessive regularization are used to prevent the overfitting, which constraint the model to capture the data well.
- 5.Features are not scaled.

Methods to avoid Overfitting:

1.Pruning: Pruning is the process of removing unnecessary branches from a decision tree to reduce complexity and prevent overfitting.

- A fully grown tree may fit the training data perfectly but perform poorly on new data.
- Pruning helps make the tree simpler, more general, and more accurate on unseen data.

2.Pre-Pruning (Early Stopping):

Stops the tree from growing too deep during construction.

The algorithm decides not to split a node further if:

- The number of samples at the node is below a threshold.
- The information gain (or splitting criterion) is too small.

Advantage: Prevents overfitting from the beginning.

Disadvantage: Might stop too early and miss important patterns.

2.Post-Pruning (Pruning after Full Growth):

First, build the full tree without restrictions.

Then, remove branches that do not improve performance on a validation dataset.

Advantage: Can capture important patterns first and then remove only unnecessary branches.

Disadvantage: Requires extra computation and validation data.

Naïve Bayes: Naive Bayes is a probabilistic classification algorithm based on Bayes' Theorem. It is called "naive" because it assumes that all features are independent of each other, which is often not true in reality but works well in practice.

Formula:

$$P(C|X) = \frac{P(X|C) \cdot P(C)}{P(X)}$$

Where:

- $P(C|X)$ = Probability of class C given features X (posterior probability)
- $P(X|C)$ = Probability of features X given class C (likelihood)
- $P(C)$ = Probability of class C (prior probability)
- $P(X)$ = Probability of features X (evidence)

Key Points:

- 1.Simple, fast, and efficient for large datasets.
- 2.Works well for text classification, spam detection, and sentiment analysis.
- 3.Assumes feature independence, which may not always hold but often gives good results.

Working:

- 1.Calculate the prior probability for each class.
- 2.Calculate the likelihood of each feature given each class.
- 3.Apply Bayes' Theorem to find the posterior probability of each class given the features.
- 4.Assign the class with the highest posterior probability to the instance.

Applications of Naive Bayes Classifier:

- 1.Spam Email Filtering:** Classifies emails as spam or non-spam based on features.
- 2.Text Classification:** Used in sentiment analysis, document categorization, and topic classification.
- 3.Medical Diagnosis:** Helps in predicting the likelihood of a disease based on symptoms.
- 4.Credit Scoring:** Evaluates creditworthiness of individuals for loan approval.
- 5.Weather Prediction:** Classifies weather conditions based on various factors.

K-Mean Clustering:

- 1.K-Means Clustering is an unsupervised machine learning algorithm that helps group data points into clusters based on their inherent similarity.
- 2.Unlike supervised learning, where we train models using labeled data, K-Means is used when we have data that is not labeled and the goal is to uncover hidden patterns or structures.
- 3.For example, an online store can use K-Means to segment customers into groups like "Budget Shoppers," "Frequent Buyers," and "Big Spenders" based on their purchase history.

Working:

- 1.Initialization:** Choose the number of clusters K and randomly select K initial centroids.
- 2.Assignment Step:** Assign each data point to the nearest centroid (using distance measure, usually Euclidean).
- 3.Update Step:** Recalculate the centroid of each cluster as the mean of all points in that cluster.
- 4.Repeat:** Continue assignment and update steps until centroids stop changing or a maximum number of iterations is reached.

Advantages:

- 1.Simple and easy to understand.
- 2.Works efficiently on large datasets.
- 3.Produces well-separated, compact clusters.

Disadvantages:

- 1.Requires choosing K in advance.
- 2.Sensitive to outliers and noisy data.
- 3.Works best for spherical clusters of similar size.

Applications:

- 1.Customer segmentation.
- 2.Image compression (color clustering).
- 3.Market basket analysis.
- 4.Document or news article clustering.

Expectation Maximization:

1.The Expectation-Maximization (EM) algorithm is a powerful iterative optimization technique used to estimate unknown parameters in probabilistic models, particularly when the data is incomplete, noisy or contains hidden (latent) variables. It works in two steps:

2.E-step (Expectation Step): Using the current parameter estimates, the algorithm calculates the expected values of the missing or hidden variables. Essentially, it assigns probabilities or "responsibilities" to different hidden outcomes given the observed data.

3.M-step (Maximization Step): With these updated expectations from the E-step, the algorithm then re-estimates the model parameters by maximizing the expected log-likelihood. This improves how well the model explains the observed data.

Working:

1.Initialization: Start with initial guesses of the model parameters.

2.E-step (Expectation step): Estimate the hidden variables (or the probability distribution over them) given the current parameters. Compute the expected value of the log-likelihood function.

3.M-step (Maximization step): Update the parameters by maximizing the expected log-likelihood found in the E-step.

4.Repeat: Alternate between E-step and M-step until convergence (i.e., parameters stop changing significantly or log-likelihood stabilizes).

Advantages:

1.Monotonic improvement: Each iteration increases (or at least never decreases) the log-likelihood.

2.Handles incomplete data well: Works effectively even with missing or hidden variables.

3.Flexibility: Can be applied to many probabilistic models, not just mixtures of Gaussians.

4.Easy to implement: The E-step and M-step are conceptually simple and often have closed-form updates.

Disadvantages:

1.Slow convergence: Convergence can be very gradual, especially near the optimum.

2.Initialization sensitive: Requires good initial parameter guesses; poor choices may yield bad solutions.

3.No guarantee of global best solution: Unlike some optimization methods, EM doesn't guarantee reaching the absolute best parameters.

4.Computationally intensive: For large datasets or complex models, repeated iterations can be costly.

Applications:

1. Clustering (Gaussian Mixture Models: a soft alternative to K-Means).
2. Image segmentation.
3. Speech recognition.
4. Medical data analysis.

Partition Around Medoids:

1. PAM is a clustering method that groups data points into cluster but instead of using centroid (average points) like K-Mean it uses actual data points called medoids to represent each cluster.
2. Medoids are actual point from the dataset so PAM is less sensitive to outlier and noises compare to the K-Means.
3. Works well where means is not a good representative.

Working:

1. **Initialization:** Select k random data points as the initial **medoids**.
2. **Assignment:** Assign each data point to the nearest medoid, forming k clusters.
3. **Update (Swap):** For each medoid m and each non-medoid point o:
 - Swap m with o and compute the total clustering cost (sum of distances).
 - If the swap reduces the cost, make o the new medoid.
4. **Repeat:** Keep performing **assignment** and **update steps** until the medoids stop changing or no swap reduces the cost further.

Advantages:

1. The main disadvantage of It is simple to understand and easy to implement.
2. K-Medoid Algorithm is fast and converges in a fixed number of steps.
3. PAM is less sensitive to outliers than other partitioning algorithms.

Disadvantages:

1. K-Medoids is not good for clustering **non-spherical or irregularly shaped groups**. This is because it chooses clusters based on how **close points are to the medoid** (compactness), not on how points are **connected** to each other.
2. It may obtain different results for different runs on the same dataset because the first k medoids are chosen randomly.

Applications of PAM:

1. Market segmentation (grouping customers by buying patterns).
2. Fraud detection (outlier handling is strong).
3. Medical data analysis (robust clustering of noisy patient data).
4. Document clustering (using similarity metrics).

Factor Analysis: Factor Analysis is a **statistical technique** used to identify hidden patterns (called *factors* or *latent variables*) that explain the relationships among a large set of observed variables.

- In **data mining**, it's used for **dimensionality reduction**: condensing many correlated variables into fewer, more meaningful factors without losing much information.
- In **data analysis**, it helps uncover the **underlying structure** of data, making it easier to interpret, visualize, and use for predictive modeling.

Working:

- 1.Input Data:** Start with many observed variables (e.g., survey responses, test scores).
- 2.Correlation Matrix:** Check how strongly variables are correlated with each other.
- 3.Extract Factors:** Identify latent factors that explain the maximum variance in the data.
- 4.Factor Loadings:** Calculate how much each variable is influenced by each factor (weights).
- 5.Rotation (optional):** Rotate factors (Varimax, Quartimax, etc.) to make interpretation easier.
- 6.Interpretation:** Name the factors based on which variables load heavily on them.

Objectives:

- 1.Reduced Dimensionality:** To reduce a large number of variables into a smaller set of factors while preserving most of the information.
- 2.Identifying Latent Variables:** To uncover hidden (unobserved) variables that explain the correlations among observed variables.
- 3.Understand Data Structures:** To analyze relationships among variables and reveal the underlying structure of the dataset.
- 4.Improve Interpretability:** To simplify complex datasets, making results easier to interpret and explain.
- 5.Pre-processing Step:** To prepare data for further analysis (e.g., regression, clustering, SEM) by reducing redundancy and multicollinearity.

Assumption: An assumption is something we believe to be true (without explicitly testing it) in order to make a method, model, or algorithm work properly. It's like a rule or condition the data or environment must satisfy for the algorithm to give meaningful or accurate results.

Working in terms of Linear Regression: Linear Regression is a statistical method that models a linear relationship between variables, assuming that errors are independent, normally distributed with constant variance, and that the predictors are not perfectly correlated.

- 1.Linearity:** Assumes a linear relationship between independent and dependent variables.
- 2.Homoscedasticity:** Assumes constant variance in the residuals (errors) across all levels of independent variables.
- 3.Normality:** Assumes the errors are normally distributed.
- 4.No Multicollinearity:** Assumes independent variables are not highly correlated with each other.
- 5.Independence:** Errors are independent.

Design Flow: Design flow in factor analysis is the systematic process of collecting data, analyzing correlations, extracting and interpreting underlying factors, which reduces many variables into a smaller number of meaningful factors.

Steps:

1.Collection of Data: Gather all the needed data from different sources like surveys, records, or databases. This is the first and most important step - without good data, no analysis will be accurate.

2.Check Correlation: Find out how variables are related to each other. Correlation shows whether one variable increases or decreases when another changes.

3.Factor Loading: Each question or variable is tested to see how strongly it connects to a common factor. A higher loading means the variable is more important to that factor.

4.Interpret the Factor: Finally, study what each factor means. Give a name or meaning to each factor based on the variables that are most related to it.

Interpreting the factors: Interpreting factors in factor analysis is the process of assigning meaning or labels to each extracted factor based on the variables that contribute most strongly to it (highest factor loadings), to understand the underlying dimensions in the data.

Steps:

Factor Loading: Examine the factor loading matrix to see how strongly each variable relates to each factor.

High Loading: Identify variables with high loadings (strongly related to the factor). These are the main contributors.

Low Loading: Identify variables with low loadings (weakly related). These may be ignored for interpretation.

Naming the Factor: Give a descriptive name to each factor based on the high-loading variables, reflecting the underlying concept.

Check Factor Scores: Analyze factor scores for each observation to see how strongly each case aligns with the identified factors.

Validation of factor analysis: Validation of factor analysis is the process of confirming that the factors identified through factor analysis are statistically reliable, conceptually meaningful, and generalizable across different samples.

Steps:

1.Check correlation: Ensure variables are sufficiently correlated; low correlations mean no common factors can be extracted.

2.Check reliability: Test internal consistency of each factor (e.g., Cronbach's alpha) to ensure factors are stable and dependable.

3.Check variance: Retained factors should explain a meaningful portion of total variance, showing they capture important patterns.

4.Cross-validation: Test the factor structure on another sample to confirm it is stable, generalizable, and not sample-specific.